José Felipe Flores da Silva

R00293192

Big Data Processing

CA1

A01 – Ex8.

There are many other publicly available transportation-related datasets out there (they can be focused on cars, buses, trains, taxis, bikes, scooters, etc).

Survey these alternative datasets, and select a dataset of interest to you:

- A publicly available transportation-related dataset (different from the Dublin Bus dataset).

Compare and contrast the new dataset w.r.t. the Dublin Bus dataset, including:

- A brief description of the dataset.
- Its main characteristics: URL for accessing to it, size, format of the data, etc.

Justify the use of a novel-use case using the aforementioned novel dataset.

- Explain the purpose, design and implementation of such novel use-case.
- For the implementation, use only Spark Core or Spark SQL. Justify your choice of technology.

Selected Dataset

Dublin Bikes station status data by Dublin City Council in CSV format. Full data set available at https://data.gov.ie/en_GB/dataset/dublinbikes-api. The "full" dataset used were 18 files holding a month's worth of data from January 2025 to June 2026 (dublin-bikes_station_status_MMYYYY.csv). The full dataset is about 1.3GB holding data of 115 stations in Dublin city centre. The micro dataset (provided with this assignment as A01_micro_dataset_8) holds the data for December 2025.

Dataset Description

The dataset has the records of the station status as snapshots of every station roughly every 5 minutes. The snapshot showcases how many bikes are available, how many docks are free, the station's name, location, and capacity.

Dataset Fields

Field	Type	Meaning
system_id	String	The bike-share system (always "dublin_bikes")
last_reported	Timestamp	Time of the snapshot (e.g. 2025-12-01 00:05:00)
station_id	Integer	Unique station identifier
num_bikes_available	Integer	Bikes available to rent
num_docks_available	Integer	Free docks to return a bike
is_installed	Boolean	Whether the station is installed
is_renting	Boolean	Whether the station allows renting
is_returning	Boolean	Whether the station accepts returns
name	String	Station name (e.g. DAME STREET)
short_name	String	Short name (empty in this dataset)
address	String	Station address
lat	Double	Station latitude
lon	Double	Station longitude
region_id	String	Region identifier (empty in this dataset)
capacity	Integer	Total docks at the station

Comparison With Dublin Bus Dataset

Both datasets are of snapshots with timestamps and GPS coordinates (latitude/longitude). The supplied Dublin Bus dataset tracks moving buses and the other tracks stationary stations' capacity. The data is captured at much more regular intervals for the Dublin Bikes at roughly every 5 minutes and per station, whereas for the Dublin Bus it's very irregular and per vehicle. The Dublin Bus dataset covers January 2013 at roughly 3GB and the Dublin Bike dataset covers a much longer timespan between January 2025 and June 2026 at roughly 1.3GB.

Novel Use Case Description

The use case came from an actual description of a need of this use case by a friend. They said that they used Dublin Bike nearly every day to go to work during summer. However, they had 3 nearby stations they could pick up a bike to go to work, but the stations wouldn't have any bikes at certain times of the day and days of the week. So, sometimes they had to walk from one station to another to try find a bike. Similarly, when dropping off their bike close to work, they had a few options of stations, but they wouldn't have any docks to drop their bikes sometimes, so they had to cycle to the next station to drop them off. What they really wanted was a live tracker of the state of the stations so they could check before committing to a station. This obviously wasn't possible within the given constraints here, but the next best thing was: data-driven probability.
The use case uses the historical snapshots to estimate, for the same day of the week and the same time of day, how likely it is that a bike will be available at each nearby pickup station and dock at each nearby drop-off station when the user arrives. It then recommends

the combination that gets the user to the destination earliest with an estimated chance of finding a bike and there being a dock to drop their bike.
This question cannot be answered with the Dublin Bus dataset. However, if the Dublin Bus dataset had a column for current passengers as well as bus capacity, it would be possible to do something very similar and predict when buses would be at capacity at each bus stop for each route at a given day of the week and time.

Novel Use Case Parameters

- My dataset directory (String: my_dataset_dir): the location where the input files are (Dublin Bike dataset). Default: "../../FileStore/tables/my_datasets/A01_micro_dataset_8/".
- Leaving at (String: leaving_at): the time when the user wants to leave their initial position. Default: "08:00".
- Day of week (Integer: day_of_week): the day of the week the user wants to take their bike. It follows Spark's dayofweek numbering (1 is Sunday and 7 is Saturday). Default: 1.
- Leaving from (Array [Double]: leaving_from): the [latitude, longitude] of origin for the journey. Default: [53.366346, -6.218066]. This matches a place I used to live nearly 10 years ago.
- Destination (Array [Double]: destination): the [latitude, longitude] of destination of the journey. Default: [53.341677, -6.260252]. This matches my friend's previous workplace.
- Average speed (Integer: avg_speed): the average cycling speed of the user in km/h. Default: 15.
- Minimum total availability (Double: min_total_availability): the minimum acceptable availability of both pick-up and drop-off. Default: 0.8.

Novel Use Case Constants

- Walking user speed: 5km/h. This is an assumption made by me. This is used to calculate how long the user will take to walk from the origin point to the pick-up station as well as how long it'll take them to walk from the drop-off point to the destination.
- Earth Radius: 6371.0km. This is the same constant used in exercise 7 to calculate distances between 2 points on the globe.
- Tolerance: 5min. This defines the tolerance around each arrival time by +-5 minutes. This is used to filter the relevant snapshots as part of the preparation.

Novel Use Case Goals

- Find the 5 closest stations to the origin and the 5 closest stations to the destination and pair them into routes.

- For each possible route, find out the arrival time at the pick-up station (departure + walking time), at the drop-off station (+ cycling time at average speed), and at the destination (+ walking time from drop-off station to destination)
- Calculate the chance of a bike using the pick-up stations' snapshots taken on the given day of the week within +- 5 minutes of the arrival time. The stations must be renting at the time to be part of the calculations. Similarly, calculate the chance of a dock using the drop-off stations' snapshots taken on the given day of the week within +-5 minutes of the arrival time at the drop-off station.
- Routes with a total chance of availability with at least the minimum total availability parameter are accepted. If the earliest-arriving acceptable route has a total chance of 1, that is the only reported route since there's 100% chance that there will be a bike to be picked up and there will be a dock to drop off the bike. If the chance isn't 100%, then the reported routes are the 5 acceptable routes sorted by arrival time at the destination, then by decreasing total chance, then by cycling distance. If no route is acceptable, report the 5 routes with the highest total chance.

Technology Choice: Spark SQL (DataFrames API)

Spark DataFrames API was the choice for this use case over Spark Core mostly because it fits the data better and lets Spark do most of the work. The data has 15 named, typed columns, including a timestamp and 3 boolean flags. Reading that with an explicit schema gives typed columns right away. Spark Core, on the other hand, would load each row as a string and would have to be parsed by position inside a map.
Spark SQL provides operators for selects, filters, group by/aggregations, max by, joins/cross joins. The whole pipeline is built from them, filtering on the day of the week and the time window, grouping by station with max by and average, joining on the station as well as cross joining the pick-up and drop-off candidates.

Default Output

Solutions with total chance of availability >= 0.8: 14

All solutions: 25

Earliest time at destination: 08:36

Latest time at destination: 08:42

Row(pickup_name='KILLARNEY STREET', dropoff_name='MOLESWORTH STREET', timeAtPickUp='08:28', timeAtDropOff='08:34', timeAtDestination='08:36', chanceOfBike=1.0, chanceOfDock=1.0, totalChance=1.0, pickupSamples=4, dropoffSamples=5)